Article Review: The No-U-Turn Sampler

Adonis Jamal
ENS Paris-Saclay
adonis.jamal@student-cs.fr

Jean-Vincent Martini
ENS Paris-Saclay
jean-vincent.martini@ens-paris-saclay.fr

Abstract

This paper analyzes Hoffman and Gelman's article, "The No-U-Turn Sampler: Adaptively Setting Path Lengths in Hamiltonian Monte Carlo" Hoffman et al. [2014], an extension of Hamiltonian Monte Carlo that eliminates manual path-length tuning by adaptively setting it during sampling. We critically evaluate the strengths, weaknesses, and practical limitations of NUTS using our own implementation. The code is available at `https://github.com/adonis107/MVA-BML`.

1 Introduction and Context

This report provides a comprehensive review of the article "The No-U-Turn Sampler: Adaptively Setting Path Lengths in Hamiltonian Monte Carlo" written by Matthew D. Hoffman and Andrew Gelman Hoffman et al. [2014]. The paper addresses a central practical limitation of Hamiltonian Monte Carlo (HMC) methods: the need to manually tune the path length parameter, which can significantly affect the efficiency of sampling. By proposing the No-U-Turn Sampler (NUTS), the authors introduce an extension to HMC that adaptively determines the path length during sampling while retaining, and in some cases improving, the efficiency of well-tuned HMC.

The context of this work is rooted in the broader field of Bayesian computation. For complex models, exact posterior inference is often intractable, necessitating the use of approximate methods. While deterministic approximations can be computationally attractive Wainwright and Jordan [2008], Markov Chain Monte Carlo (MCMC) methods remain appealing because they are asymptotically unbiased and broadly applicable Neal [1993]. However, traditional algorithms such as random-walk Metropolis Metropolis et al. [1953] and Gibbs sampling Geman and Geman [1984] suffer from inefficient exploration in high-dimensional or highly correlated parameter spaces. HMC addresses these issues by introducing auxiliary momentum variables and simulating Hamiltonian dynamics to propose new states, allowing for larger and more informed jumps in the parameter space Neal et al. [2011]. Despite its advantages, HMC requires the user to specify two sensitive parameters: the step size and the number of leapfrog steps (or equivalently, the integration time). While adaptive methods have been developed to mitigate this issue Andrieu and Thoms [2008], Nesterov [2009], it remains a significant barrier that limits the accessibility and widespread adoption of HMC in general-purpose software Hoffman et al. [2014].

The NUTS algorithm eliminates the need to specify the number of leapfrog steps by dynamically building a set of candidate states along the Hamiltonian trajectory and stopping automatically when the simulated path begins to double back on itself, hence the name "No-U-Turn". In addition, the authors introduce a primal–dual averaging scheme to adapt the step size during a warm-up phase. Together, these innovations yield a nearly tuning-free algorithm that preserves the theoretical strengths of HMC while dramatically improving usability.

In this report, we first examine the theoretical, computational, and empirical methods developed in the paper, detailing how NUTS modifies and extends standard HMC. We then critically assess the strengths and limitations of the approach, both from a theoretical and practical perspective thanks to our own implementation.

2 The No-U-Turn Sampler: Methodology and Analysis

Problem Definition and Limitations of HMC. HMC Neal et al. [2011], Duane et al. [1987] introduces auxiliary momentum variables $r \in \mathbb{R}^D$ paired with the parameters of interest $\theta \in \mathbb{R}^D$, defining the joint distribution $p(\theta, r) \propto \exp\left\{\mathcal{L}(\theta) - \frac{1}{2} r \cdot r\right\}$, where $\mathcal{L}(\theta)$ is the log-joint density up to a normalising constant. Each iteration resamples $r \sim \mathcal{N}(0, I)$ and simulates Hamiltonian dynamics via the leapfrog integrator for L steps of size ϵ, producing a proposal accepted or rejected via a Metropolis step. Because the leapfrog integrator is volume-preserving, proposals can be simultaneously distant and highly probable, enabling efficient exploration of high-dimensional targets.
However, as mentioned earlier, HMC's performance depends critically on both ϵ and L. Too large an ϵ yields high rejection rates and too small wastes computation. If L is too small the chain degenerates into a random walk, while if L is too large the trajectory loops back on itself, wasting gradient evaluations. Selecting a good L typically requires several costly preliminary runs and considerable expertise, limiting HMC's adoption in general-purpose inference software Hoffman et al. [2014].

The No-U-Turn Criterion and Trajectory Construction. NUTS eliminates the need to specify L by stopping trajectory simulation automatically when further simulation would no longer move the proposal away from the starting point. The key quantity is the time derivative of half the squared distance between current position $\tilde{\theta}$ and initial position θ. When this derivative becomes negative, the trajectory is doubling back, making further simulation inefficient. A naive stopping rule based on this criterion would violate time-reversibility, so NUTS instead builds trajectories both forwards and backwards in fictitious time via a recursive doubling scheme.
At each doubling step j, a direction $v_j \sim \mathrm{Uniform}(\{-1, +1\})$ is drawn and 2^j leapfrog steps of size $v_j \epsilon$ are taken, implicitly constructing a balanced binary tree whose leaves are visited position-momentum states. To ensure detailed balance, the model is augmented with a slice variable $u \sim \mathrm{Uniform}([0, \exp\{\mathcal{L}(\theta) - \frac{1}{2} r \cdot r\}])$, and the candidate set $\mathcal{C} \subseteq \mathcal{B}$ is restricted to visited states lying within the slice. Doubling halts either upon numerical instability or when a U-turn is detected in any balanced subtree, i.e., when the leftmost and rightmost leaves (θ^-, r^-), (θ^+, r^+) satisfy

$$(\theta^+ - \theta^-) \cdot r^- < 0 \quad \text{or} \quad (\theta^+ - \theta^-) \cdot r^+ < 0.$$

States added during a halting iteration are excluded from $\mathcal{C}$ to preserve detailed balance. The efficient implementation further reduces memory from $O(2^j)$ to $O(j)$ by propagating a single representative state through the recursion and applying a Metropolis-Hastings accept/reject step after each doubling rather than sampling from $\mathcal{C}$ explicitly at the end.

Adaptive Step Size Selection via Dual Averaging. To also automate the tuning of ϵ, the authors adapt the primal-dual averaging algorithm of Nesterov [2009]. The step size is only updated during a warm-up phase to drive the average acceptance statistic towards a target δ, via:

$$x_{t+1} = \mu - \frac{\sqrt{t}}{\gamma(t + t_0)} \sum_{i=1}^{t} H_i, \qquad \bar{x}_{t+1} = t^{-\kappa} x_{t+1} + (1 - t^{-\kappa}) \bar{x}_t,$$

where $x = \log \epsilon$, H_t is a custom statistic that describes the behavior of an MCMC algorithm at iteration $t \geq 1$ (possibly a function of the acceptance rate of the Metropolis step), $\mu = \log(10\epsilon_0)$ encourages testing larger values of ϵ, and γ, t_0, κ are fixed hyperparameters set to 0.05, 10, and 0.75 throughout. This scheme renders NUTS entirely tuning-free.

Some components are only briefly sketched in the original paper, so we provide more detailed derivations here. We focus on the two key theoretical ingredients ensuring correctness of NUTS: the uniform distribution over the candidate set $\mathcal{C}$ and the memory-efficient transition kernel.

Rigorous Proof of Uniform Distribution over the Candidate Set. In the original paper, the uniformity of the conditional distribution over the candidate set $\mathcal{C}$ (Equation (2) in the paper) is derived very concisely from conditions C.1 to C.4. The argument is compressed into a few proportionality statements. We restate it more explicitly.
Let $w = (\theta, r)$. The augmented joint density is:

$$p(\theta, r, u) \propto \mathbb{I}\left[u \leq \exp\left(L(\theta) - \tfrac{1}{2} r \cdot r\right)\right].$$

Conditioning on $(u, \mathcal{B}, \mathcal{C}, \varepsilon)$ gives:

$$p(w \mid u, \mathcal{B}, \mathcal{C}, \varepsilon) \propto p(\mathcal{B}, \mathcal{C} \mid w, u, \varepsilon)\, p(w \mid u). \tag{1}$$

First, by definition of the slice, $p(w \mid u) \propto \mathbb{I}\big[u \leq \exp\big(L(\theta) - \frac{1}{2} r \cdot r\big)\big]$. Then, Condition C.3 ensures that $w \in \mathcal{C}$ implies the slice constraint holds. Hence on $\mathcal{C}$ the indicator equals 1, and $p(w \mid u) \propto 1 \quad \text{for } w \in \mathcal{C}$.
Second, by C.2 the current state always belongs to $\mathcal{C}$, and by C.4 the probability of generating the same $(\mathcal{B}, \mathcal{C})$ is identical for any $w \in \mathcal{C}$. Therefore, $p(\mathcal{B}, \mathcal{C} \mid w, u, \varepsilon) \propto \mathbb{I}[w \in \mathcal{C}]$.
Substituting into (1) yields $p(w \mid u, \mathcal{B}, \mathcal{C}, \varepsilon) \propto \mathbb{I}[w \in \mathcal{C}]$, which shows that the conditional distribution is uniform over $\mathcal{C}$. Finally, by C.1 (volume preservation of leapfrog), elements of $\mathcal{C}$ can be treated as having equal probability mass, so the normalized distribution is

$$p(w \mid u, \mathcal{B}, \mathcal{C}, \varepsilon) = \frac{1}{|\mathcal{C}|}\, \mathbb{I}[w \in \mathcal{C}],$$

as required.

The Transition Kernel and Proof of Detailed Balance. We now turn to the second key ingredient ensuring correctness of Efficient NUTS, the memory-efficient transition kernel, whose role is to preserve the uniform distribution over the candidate set $\mathcal{C}$ while enabling incremental sampling from newly constructed subtrees. To reduce memory complexity from $\mathcal{O}(2^j)$ to $\mathcal{O}(j)$, Efficient NUTS introduces a custom transition kernel $T(w'|w, \mathcal{C})$. Instead of uniformly sampling the full candidate set $\mathcal{C}$, the algorithm partitions $\mathcal{C}$ into $\mathcal{C}^{old}$ (previously visited states) and $\mathcal{C}^{new}$ (states added during the final doubling). For a current state $w \in \mathcal{C}^{old}$, it proposes a jump to $w' \in \mathcal{C}^{new}$ uniformly and accepts it with probability $\min\left(1, \frac{|\mathcal{C}^{new}|}{|\mathcal{C}^{old}|}\right)$. This allows incremental sampling from subtrees without storing all states, while favoring exploration of distant regions.
The authors state that this kernel satisfies detailed balance with respect to the uniform distribution $p(w|\mathcal{C}) = \frac{1}{|\mathcal{C}|}$, but omit the exact proof. We provide a brief derivation here.
For $w \in \mathcal{C}^{old}$ and $w' \in \mathcal{C}^{new}$, the detailed balance condition requires:

$$p(w|\mathcal{C})T(w'|w, \mathcal{C}) = p(w'|\mathcal{C})T(w|w', \mathcal{C}) \tag{2}$$

Substituting the respective proposal and acceptance probabilities yields:

$$\frac{1}{|\mathcal{C}|} \frac{1}{|\mathcal{C}^{new}|} \min\left(1, \frac{|\mathcal{C}^{new}|}{|\mathcal{C}^{old}|}\right) = \frac{1}{|\mathcal{C}|} \frac{1}{|\mathcal{C}^{old}|} \min\left(1, \frac{|\mathcal{C}^{old}|}{|\mathcal{C}^{new}|}\right)$$

Multiplying both sides by $|\mathcal{C}||\mathcal{C}^{old}||\mathcal{C}^{new}|$ simplifies the expression to:

$$|\mathcal{C}^{old}| \min\left(1, \frac{|\mathcal{C}^{new}|}{|\mathcal{C}^{old}|}\right) = |\mathcal{C}^{new}| \min\left(1, \frac{|\mathcal{C}^{old}|}{|\mathcal{C}^{new}|}\right) \Rightarrow \min(|\mathcal{C}^{old}|, |\mathcal{C}^{new}|) = \min(|\mathcal{C}^{new}|, |\mathcal{C}^{old}|)$$

Since both sides are trivially equal, detailed balance holds, proving that the memory-efficient transition kernel preserves the target uniform distribution over $\mathcal{C}$.

Empirical Evaluation The authors evaluate NUTS and HMC on four targets: a 250-dimensional multivariate normal, the posterior of a Bayesian logistic regression (25 parameters) and its hierarchical extension (302 parameters) both fitted to the German credit data set, and a 3001-dimensional stochastic volatility model fitted to S&P 500 returns. Performance is measured as effective sample size (ESS) per gradient evaluation, with the minimum ESS across dimensions reported. HMC is tested across a grid of simulation lengths and target acceptance rates, while NUTS is tested across values of δ only, for a total of over 3800 experiments.
The dual averaging scheme reliably matches the realised acceptance statistic to its target δ and $\bar{\epsilon}$ converges within a few hundred warm-up iterations on most models. NUTS trajectory lengths concentrate on integer powers of two, confirming that the U-turn criterion is typically triggered only after a complete doubling. In terms of ESS per gradient, NUTS matches or exceeds the best tuned HMC across all four distributions, outperforming it by roughly a factor of two on the multivariate normal and 1.5 on the stochastic volatility model, without requiring any tuning of the trajectory length.

3 Critical Evaluation and Implementation

Strengths and Weaknesses of the Article. The major strength of the article is that it resolves a central practical limitation of HMC, the manual tuning of the trajectory length, through a conceptually simple yet theoretically rigorous construction. The No-U-Turn criterion and recursive doubling preserve detailed balance, and dual averaging for step-size adaptation makes the method essentially tuning-free. However, the dual-averaging hyperparameters are justified empirically rather than theoretically, leaving open the question of whether they remain robust across posterior geometries significantly different from those tested. Performance guarantees are mainly asymptotic, and the experiments cover a limited range of posterior geometries, leaving the behavior on more pathological targets unexplored, a gap our stress tests begin to address.

Implementation Validation. We implemented HMC and NUTS with dual averaging from scratch in Python and reproduced all four original benchmarks: 250-d multivariate normal, Bayesian logistic regression (25 parameters), its hierarchical extension (302 parameters), and 3001-d stochastic volatility. The dual-averaging scheme reliably drives the acceptance statistic towards δ within a few hundred warm-up iterations, and NUTS trajectory lengths concentrate on powers of two, confirming the U-turn criterion triggers at complete doublings. Our discrepancy plots ($h - \delta$ across δ) closely match Figure 3 of the original paper: NUTS matches or exceeds the best-tuned HMC in ESS per gradient on all four models without trajectory-length tuning.

Beyond the Paper: Stress Tests on Complex Posteriors. The original evaluation is limited to unimodal or mildly structured targets. We design three experiments targeting pathologies common in modern Bayesian models: **(i)** extreme multimodality, **(ii)** dense correlations, and **(iii)** spatially varying geometry.

(i) Multimodal Stress Test: Bayesian Neural Network. We fit a two-hidden-layer BNN (30→5→5→1, 191 parameters, $\mathcal{N}(0, 100)$ priors) to the UCI Breast Cancer dataset. Weight-permutation symmetries create $5! \times 5! = 14{,}400$ equivalent modes separated by saddle points. With $\delta = 0.65$, $M = 450$, $M_{\text{adapt}} = 150$, NUTS adapts trajectory lengths between 1 and 64 leapfrog steps, exploring a wider region of weight space than HMC. Trace plots confirm NUTS covers a broader range of weight values, while HMC exhibits prolonged stagnation, suggesting confinement to a single mode. However, our results expose two underappreciated limitations. First, NUTS does not always achieve higher ESS per gradient than well-tuned HMC: in our tests, NUTS yields 0.03×10^{-4} ESS/grad versus 1.57×10^{-4} for HMC. On posteriors with narrow correlated valleys, NUTS continues doubling its tree, technically avoiding a U-turn, while the trajectory wanders inefficiently. A fixed-L HMC interrupts this behaviour at a fraction of the cost. Second, the paper measures efficiency solely in ESS per gradient evaulation, which conceals NUTS's algorithmic overhead: each step requires recursive `build_tree` calls, forward/backward state allocation, and per-node dot products to check the U-turn condition, whereas HMC executes a simple leapfrog loop. Even when NUTS matches HMC in ESS/grad, well-tuned HMC is often faster in wall-clock time.

(ii) Dense Correlation Test: GP Classification. We apply GP classification with an RBF kernel ($\ell = 1$, $\sigma_f^2 = 1$) to 150 points from the California Housing dataset, yielding a fully dense 150×150 covariance K. The gradient $\nabla \log p(\mathbf{f}) = -K^{-1}\mathbf{f} + \mathbf{y} \odot \sigma(-\mathbf{y} \odot \mathbf{f})$ requires a dense solve at every leapfrog step. With $M = 500$ and $M_{\text{adapt}} = 250$, NUTS achieves 2.14×10^{-3} ESS/grad at $\delta = 0.65$ versus 1.52×10^{-3} for HMC, a 40% improvement. However, NUTS consumed roughly $4\times$ more gradient evaluations in total (24,840 vs. 6,388), as the dense covariance structure induces long trajectories before a U-turn is detected. This suggests that on densely correlated targets, NUTS's adaptive mechanism partially trades absolute gradient cost for better per-gradient sample quality, and the practical advantage over a well-tuned HMC is narrower than on other benchmarks. Note that since both methods invoke the same dense solve per leapfrog step, gradient evaluation costs are identical here, so ESS/grad faithfully reflects wall-clock efficiency.

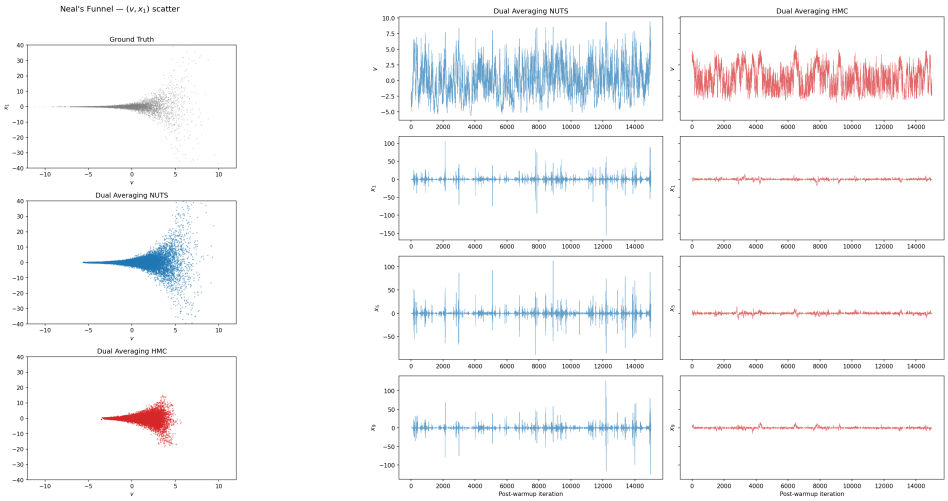
(iii) Neal's Funnel. Neal's funnel Neal [2003] is a canonical pathological target: $v \sim \mathcal{N}(0, 9)$, $x_i \mid v \sim \mathcal{N}(0, e^v)$ for $i = 1, \ldots, 9$. As $v \to -\infty$, the conditional variance $e^v \to 0$ creates a microscopic neck demanding tiny steps, while the wide top ($v > 0$) requires large steps. HMC's fixed step size cannot handle both regimes. Figure 1a shows that NUTS covers the full funnel including negative v, closely matching the ground truth, whereas HMC is confined to $v \in [0, 4]$ and completely misses the funnel neck. The trace plots (Figure 1b) confirm that NUTS transitions freely between both

regimes while HMC gets stuck with long flat segments indicating rejected proposals. Quantitatively, NUTS recovers $\mathrm{Var}(v) \approx 9$ (the true value), while HMC severely underestimates it.

Summary. Across all three stress tests, NUTS consistently outperforms HMC without trajectory-length tuning. The gains are most dramatic when the posterior geometry varies spatially (Neal's funnel) or features many equivalent modes (BNN), precisely the scenarios unexplored in the original paper, confirming that NUTS's adaptive mechanism is essential for complex posteriors. The full code and results for these experiments are available in our GitHub repository.

4 Conclusion

This review confirms that NUTS constitutes a significant advance in practical Bayesian computation. It is tuning-free and avoids the hassle of finding L for HMC, while also handling varying geometry. Our independent implementation validates the original results: NUTS matches or exceeds the best-tuned HMC on all four benchmarks while eliminating trajectory-length tuning. More importantly, our stress tests on a multimodal BNN, a densely correlated GP classifier, and Neal's funnel demonstrate that the adaptive trajectory length is critical on posteriors with complex geometries. HMC fails catastrophically on the funnel and stagnates in the multimodal landscape, whereas NUTS manages to handle, and achieves modest but consistent gains on even densely correlated targets where its advantage is more limited, confirming its suitability as a general-purpose inference engine for modern Bayesian models.



(a) Neal's Funnel (v, x_1) scatter. **Top:** ground truth. **Centre:** NUTS covers the full funnel. **Bottom:** HMC fails to enter the narrow neck ($v < 0$).

(b) Trace plots. NUTS (left, blue) transitions freely across the full v range with good mixing in all x_i. HMC (right, red) gets trapped with flat segments and x_i degenerating to near-zero.

Figure 1: Diagnostics on Neal's Funnel.

References

Matthew D Hoffman, Andrew Gelman, et al. The no-u-turn sampler: adaptively setting path lengths in hamiltonian monte carlo. *J. Mach. Learn. Res.*, 15(1):1593–1623, 2014.

Martin J Wainwright and Michael I Jordan. Graphical models, exponential families, and variational inference. *Foundations and Trends® in Machine Learning*, 1(1-2): 1–305, 2008.

Radford M Neal. Probabilistic inference using markov chain monte carlo methods. 1993.

Nicholas Metropolis, Arianna W Rosenbluth, Marshall N Rosenbluth, Augusta H Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The journal of chemical physics*, 21(6):1087–1092, 1953.

Stuart Geman and Donald Geman. Stochastic relaxation, gibbs distributions, and the bayesian restoration of images. *IEEE Transactions on pattern analysis and machine intelligence*, (6):721–741, 1984.

Radford M Neal et al. Mcmc using hamiltonian dynamics. *Handbook of markov chain monte carlo*, 2(11):2, 2011.

Christophe Andrieu and Johannes Thoms. A tutorial on adaptive mcmc. *Statistics and computing*, 18(4):343–373, 2008.

Yurii Nesterov. Primal-dual subgradient methods for convex problems. *Mathematical programming*, 120(1):221–259, 2009.

Simon Duane, Anthony D Kennedy, Brian J Pendleton, and Duncan Roweth. Hybrid monte carlo. *Physics letters B*, 195(2):216–222, 1987.

Radford M Neal. Slice sampling. *The Annals of Statistics*, 31(3):705–767, 2003.